

NARADA: The Scanning Agent, Explained

How NARADA decides where to point the receiver, how it keeps learning while a mission runs, and what it achieves. Companion to *The RF Environment, Explained*, which describes the world NARADA operates in.

1. What NARADA is

NARADA is a Recurrent PPO agent with LSTM memory. Every turn it receives a summary of what the receiver has heard so far, and it picks one of the 36 frequency bands to listen to next.

We treat scanning as a **Partially Observable Markov Decision Process (POMDP)**: the receiver can only ever see the one band it is tuned to, so NARADA has to decide from partial observations, never from complete emitter information.

Three properties define it:

- **It has memory.** A decision can depend on something the receiver heard many seconds earlier.
- **It needs no prior intelligence.** No emitter library, no pre-mission frequency list. Every mission starts from zero knowledge.
- **It keeps adapting.** Hit/miss feedback continuously updates the policy, including after deployment.

2. How one decision is made

Action. Choose one of the 36 bands to scan next.

Observation. Built only from the receiver's own scan history, for each band:

Feature	What it tells NARADA
hit_rate	how often looking here has produced a detection
visit_density	how much of the listening time this band has had
staleness	how long since this band was last checked
measured_dbm	strength of the last signal heard here
pulse_width	pulse width of the last signal heard here
aoa_sin, aoa_cos	bearing of the last signal heard here
pulse_count	how many pulses the last detection contained
hit_streak	how many consecutive visits here have produced a hit
band_priority	operator-assigned importance of this band

plus the current band, the mission clock, the current hit streak and, in the latest layout, the reward from the previous step.

Choice. NARADA outputs a preference over all 36 bands and **samples** a band from it rather than always taking the single top choice. This keeps its scanning diverse and its coverage wide — the fix for the camping behaviour described in section 7.

3. Why memory matters

The right band to check depends on the *history* of the mission: which bands were checked, what came back, and how long ago. The LSTM carries that history from one decision to the next and resets cleanly at the start of every mission.

This gives NARADA the ability to adapt **inside a single mission, without changing any weights**. Dropped into a completely unfamiliar spectrum, it found new emitters at a rising rate — **18 to 20 new emitters every 30 seconds** once it had worked out where the traffic was, with significant policy change visible from around **90 seconds**.

We confirmed the value of memory with a controlled comparison: the same training setup, same observation, same reward, same budget — only the architecture changed. The recurrent version caught **35% more of the transmitted traffic** than a memoryless PPO policy, winning on interception ratio in **83% of paired missions**.

4. How NARADA learns online

NARADA runs a closed loop: pick a band, hear hit or miss, update its belief about all 36 bands, pick the next one — every dwell, 50 to 100 ms.

Two things update, on two timescales:

	Updates	What changes
LSTM memory	every decision	NARADA's picture of the current spectrum
Model weights	every 8,192 decisions (about 9 minutes on an NVIDIA GTX 1650)	NARADA's scanning strategy itself

Missions are run as a continuous stream so that each weight update draws on a full batch of experience. Over roughly an hour of continuous operation in a new environment, the weight updates turn NARADA's in-mission improvisation into a standing skill it applies from the very first second of the next mission.

The reward is a fixed formula computed only from what the receiver itself reports:

$$R = 0.5 \cdot h + (1.5 - d) \cdot s + 0.5 \cdot (\text{declared hits}) - 3.0 \cdot d \cdot n$$

Term	Role
$0.5 \cdot h$	exploit what pays — bands with a good hit rate
$(1.5 - d) \cdot s$	explore what is overdue — stale bands that have had little time
$0.5 \cdot (\text{declared hits})$	reward for detections on this look

Term	Role
$3.0 \cdot d \cdot n$	cost of hogging airtime — stops NARADA settling on one band

h = hit rate, d = airtime share, s = staleness, n = slots in the dwell. Every quantity is read from NARADA's own observation, so the same learning works in the field. Before any training, the reward is screened against known good and bad scanning policies to make sure it ranks them correctly.

5. Operator-guided priority

Every ES system already carries a threat library. NARADA can take a **per-band priority input** from the operator or mission intel. Priority is part of NARADA's observation, and the reward adds a bonus for discovering new emitters on priority bands — so the scanning strategy can be guided toward high-threat bands, and a band can be de-prioritised again once its emitter is out of action.

The human is **on** the loop, not **in** it: NARADA works fully autonomously, and intel is used when it is available.

6. How NARADA handles different emitter types

Emitter type	How NARADA handles it
Periodic	stochastic policy sampling, so the scan never locks into a fixed rhythm an emitter's cycle can fall between
Spatially scanning	bearing (angle of arrival) of each detection is part of the observation, across repeated looks
Frequency agile	continuous reallocation of listening time across bands as activity moves; tested on synthetic frequency-hopping worlds

7. Challenges we solved

Challenge	Solution
Dense-band camping looked good. Parking on one busy band scored well on interception ratio while missing other emitters.	We score every result on interception ratio, censored intercept time and coverage together, so ignoring emitters is always penalised.
The model over-prioritised camping. Always taking the highest-probability band made it stick to one band.	Sampling from the policy distribution gave diverse band selection and much wider coverage, with no retraining.
Limited band exploration. The model found it could alternate between two bands to collect both exploration and hit rewards.	Visit density and staleness in the reward now encourage less-visited bands, removing that periodic behaviour.

Challenge	Solution
Empty bands created a bias. 12 of 36 bands carried no traffic in the dataset.	The online PPO adaptation path lets NARADA re-learn when the real spectrum differs, and we evaluate it with scenario variation.

8. Results

Every scheduler was run on the identical environment, the identical 47 scenarios and the identical noise (141 missions), and compared mission by mission. Reward function: `reward_balance`.

Scheduler	Avg reward	Interception ratio	Censored intercept time	Emitter coverage	Avg intercept rate
Round robin	220	0.059	3.72 s	0.877	1.060 /s
Recency	290	0.112	2.73 s	0.911	1.109 /s
NARADA 23a	568.2	0.153	1.90 s	0.925	1.144 /s

NARADA leads on every column of the table. Mission by mission, it beats the round-robin sweep on **both** headline objectives in **83.7%** of missions, and the recency heuristic in **73.8%**. On the Pareto chart it sits alone in the top-left corner: every other scheduler trades interception ratio against speed.

9. What it takes to run

Parameters	3.8 million
Size on disk	43.7 MB
Time per decision	2.5 ms on a single CPU thread, no GPU
Time available per decision	50 ms

NARADA decides about **twenty times faster than real time** on one CPU core. Training ran on a single consumer laptop GPU.

NARADA is built on Stable-Baselines3 2.9 and SB3-Contrib 2.9 (Recurrent PPO with LSTM), PyTorch 2.14 and Gymnasium 1.3. Figures are from the project's own runs: the 141-mission comparison, the matched PPO-versus-LSTM comparison, the in-mission adaptation test, and CPU timing of the trained policy.